Computational Economics

# Lecture: Policy Function Iteration (PFI)

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

# Lecture Overview

- **Policy Evaluation**
  - The policy evaluation operator
  - Repeated updates and a linear system
- **Policy Function Iteration**
  - Evaluation and improvement in the maze
  - The algorithm and its optimality criterion
- **Computational Choices**
  - Generalized and modified policy iteration
  - Computational cost and continuous states

# From Value Iteration to Policy Iteration

VFI chooses the best action at every state in each update:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \{r(s,a) + \beta \mathbb{E}[V_k(s') \mid s, a]\}.$$

We can separate the computation into two operations:

- **Policy evaluation**: hold $\sigma$ fixed and compute its value $V^\sigma$.
- **Policy improvement**: given $V^\sigma$, choose a policy greedy with respect to that value:

$$\widetilde{\sigma}(s) \in \arg\max_{a \in \mathcal{A}(s)} \{r(s,a) + \beta \mathbb{E}[V^\sigma(s') \mid s, a]\}.$$

**Policy function iteration (PFI)** alternates these operations:

$$\sigma_k \xrightarrow{\text{policy evaluation}} V^{\sigma_k} \xrightarrow{\text{policy improvement}} \sigma_{k+1} \longrightarrow \cdots$$

We use stationary deterministic policies, with $0 < \beta < 1$ and $|r(s,a)| \leq \bar{r} < \infty$.

# The Policy Evaluation Operator

For a fixed policy $\sigma$, the **policy evaluation operator** $T_\sigma$ maps a candidate continuation-value function $V$ to an updated function:

$$(T_\sigma V)(s) = r(s, \sigma(s)) + \beta \mathbb{E}[V(s') \mid s, a = \sigma(s)].$$

- The action is supplied by $\sigma(s)$; this update does not choose a new action.
- The input $V$ may be any bounded candidate function. It need not equal $V^\sigma$.

**Policy evaluation** means finding the fixed point:

$$V^\sigma = T_\sigma V^\sigma.$$

One application $T_\sigma V$ is one update; repeated updates compute the policy's value.

# Iterative Policy Evaluation

Hold $\sigma$ fixed. Choose $V_0$, and update all states using the same previous function:

$$V_{j+1}(s) = (T_\sigma V_j)(s) = r(s, \sigma(s)) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s, \sigma(s)) V_j(s').$$

The index $j$ counts evaluation updates, not changes of policy.

As with the Bellman optimality operator, discounting gives a contraction:

$$\|T_\sigma V_1 - T_\sigma V_2\|_\infty \leq \beta \|V_1 - V_2\|_\infty.$$

Hence $V_j \to V^\sigma$. For a target evaluation error $\varepsilon_{\text{eval}} > 0$, compute

$$d_j = \|V_{j+1} - V_j\|_\infty, \qquad \|V_j - V^\sigma\|_\infty \leq \frac{d_j}{1 - \beta}.$$

Stop and return $V_j$ when $d_j \leq (1 - \beta)\varepsilon_{\text{eval}}$.

# Policy Evaluation as a Linear System

For finitely many states, collect the rewards and transition probabilities selected by $\sigma$:

$$r^\sigma(s) = r(s, \sigma(s)), \qquad P^\sigma(s, s') = P(s' \mid s, \sigma(s)).$$

Each row of $P^\sigma$ describes the next-state probabilities under the policy's chosen action.

The policy's Bellman equation becomes

$$V^\sigma = r^\sigma + \beta P^\sigma V^\sigma \qquad \Longleftrightarrow \qquad (I - \beta P^\sigma) V^\sigma = r^\sigma.$$

- $I$ is the identity matrix. There is one equation and one unknown value per state.
- Solve this linear system directly, or use repeated evaluation updates.
- A direct solve avoids an inner iteration; its cost grows with the number of states.

# Example: Evaluating the Maze Policy

Use the same policy $\sigma$ as before, now with $\beta = 0.9$.

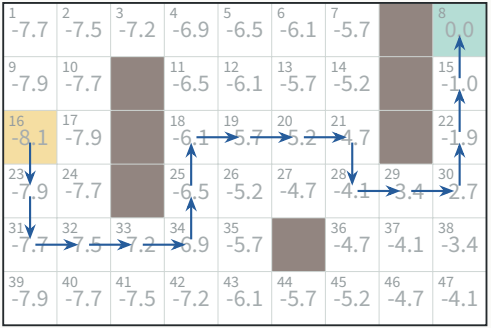



Figure 1: Discounted values under $\sigma$.

- Each move gives $-1$, including entry into cell 8; rewards afterward are zero.
- At cell 25, the policy chooses U to cell 18.

$$V^\sigma(8) = 0,$$
$$V^\sigma(25) = -1 + 0.9V^\sigma(18)$$
$$\approx -6.513.$$

At the starting cell, $V^\sigma(16) \approx -8.147$.

Cell values are rounded to one decimal place.

# Example: Improving and Re-evaluating the Policy

Change only cell 25 to R: $\widetilde{\sigma}(25) = \mathrm{R}$; keep $\widetilde{\sigma}(s) = \sigma(s)$ elsewhere.

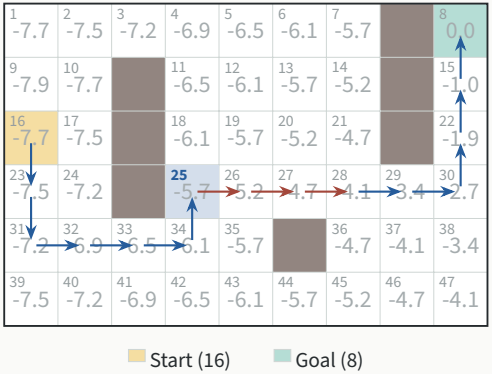



Figure 2: Re-evaluated values under $\widetilde{\sigma}$.

**Policy improvement using $V^{\sigma}$:**

$$Q^{\sigma}(25, \mathrm{U}) \approx -6.513,$$
$$Q^{\sigma}(25, \mathrm{R}) \approx -5.695.$$

**Policy evaluation for the new policy:**

$$V^{\widetilde{\sigma}}(25) \approx -5.695,$$
$$V^{\widetilde{\sigma}}(16) \approx -7.712.$$

The shortcut also improves values at states whose paths pass through cell 25.

# Policy Function Iteration (PFI) Algorithm

0. **Input.** Finite state set $\mathcal{S}$, finite nonempty feasible action sets $\mathcal{A}(s)$, rewards $r(s,a)$, transition probabilities $P(s' \mid s,a)$, and discount factor $\beta$.
1. **Initialize.** Choose a feasible policy $\sigma_0(s) \in \mathcal{A}(s)$ for every state. Set $k = 0$.
2. **Policy evaluation.** Form $r^{\sigma_k}$ and $P^{\sigma_k}$, then solve

$$(I - \beta P^{\sigma_k}) V^{\sigma_k} = r^{\sigma_k}.$$

3. **Compute action values.** For every feasible $(s,a)$, compute

$$Q^{\sigma_k}(s,a) = r(s,a) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s,a) V^{\sigma_k}(s').$$

   Use this same evaluated $V^{\sigma_k}$ for every action comparison.

# PFI Algorithm (continued)

4. **Policy improvement.** For every state, select

$$\sigma_{k+1}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^{\sigma_k}(s, a).$$

   Keep $\sigma_k(s)$ if it is already a maximizer; otherwise use a fixed action order to break ties.

5. **Check the policy.**
   - If $\sigma_{k+1}(s) = \sigma_k(s)$ at every state, continue to **Step 6**.
   - Otherwise, set $k \leftarrow k + 1$ and return to **Step 2**.

6. **Return and stop.** Output the policy and its evaluated value:

$$\sigma^* = \sigma_k, \qquad V^* = V^{\sigma_k}.$$

The exact optimality statement uses exact policy evaluation. A numerical solve introduces rounding error; truncated evaluation introduces an additional approximation.

# Policy Improvement and Optimality

Every greedy improvement satisfies the policy improvement theorem:

$$Q^{\sigma_k}(s, \sigma_{k+1}(s)) \geq V^{\sigma_k}(s) \quad \forall s \quad \Longrightarrow \quad V^{\sigma_{k+1}}(s) \geq V^{\sigma_k}(s) \quad \forall s.$$

When the policy does not change, it is greedy with respect to its own value:

$$V^{\sigma_k}(s) = \max_{a \in \mathcal{A}(s)} Q^{\sigma_k}(s, a) = (TV^{\sigma_k})(s).$$

Thus $V^{\sigma_k} = V^*$, by uniqueness of the Bellman optimality fixed point.

- Retaining the current action in a tie prevents changes between equally good actions.
- If the policy changes, its value strictly increases at some state.
- With finitely many deterministic policies, exact PFI therefore terminates at an optimal policy after finitely many improvements.

# Generalized Policy Iteration

**Generalized policy iteration (GPI)** lets policy evaluation and policy improvement interact, allowing evaluation to stop before convergence.

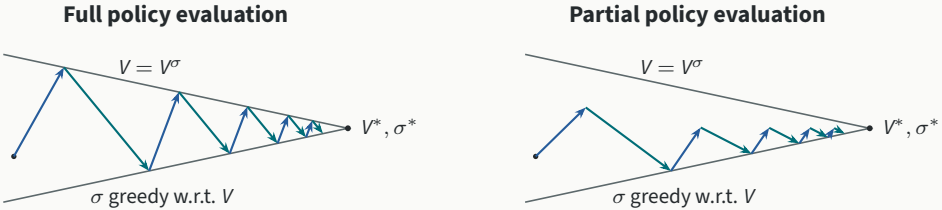



Figure 3: Full and partial policy evaluation within GPI (schematic).

**Policy evaluation** moves toward $V = V^\sigma$. **Policy improvement** makes $\sigma$ greedy with respect to $V$.

At the intersection, both conditions hold: $V = T_\sigma V = TV$, so $V = V^*$ and $\sigma$ is optimal.

Concept adapted from Sutton and Barto (2018), Section 4.6, pp. 86–87.

# Modified Policy Iteration

In GPI, the evaluation arrows may stop short of full consistency. **Modified policy iteration** implements this using a fixed number $m \geq 1$ of policy evaluation updates between policy improvements.

1. **Policy improvement.** Choose a policy greedy with respect to the current candidate $V_k$:

$$\sigma_{k+1}(s) \in \arg\max_{a \in \mathcal{A}(s)} \{r(s,a) + \beta \mathbb{E}[V_k(s') \mid s, a]\}.$$

2. **Policy evaluation.** Hold $\sigma_{k+1}$ fixed. Set $V_{k,0} = V_k$, then compute

$$V_{k,j+1} = T_{\sigma_{k+1}} V_{k,j}, \qquad j = 0, \ldots, m-1.$$

3. Set $V_{k+1} = V_{k,m}$ and repeat.

$V_k$ is an approximate value, so an unchanged greedy policy alone does not certify optimality. Check the Bellman residual $\|TV_k - V_k\|_\infty$, as in VFI.

## Choosing the Amount of Policy Evaluation

**One policy evaluation update: VFI.** Since $\sigma_{k+1}$ is greedy with respect to $V_k$,

$$T_{\sigma_{k+1}} V_k = TV_k.$$

Setting $m = 1$ therefore gives $V_{k+1} = TV_k$.

**Several policy evaluation updates: modified policy iteration.** Extra updates reuse the chosen actions. They avoid a new maximization while refining the value estimate.

**Full policy evaluation: PFI.** Continuing evaluation to its fixed point gives $V^{\sigma_{k+1}}$; the next policy improvement uses that policy's value.

For a finite MDP under our discounted, bounded-reward assumptions, these exact updates converge to $V^*$, including any fixed $m \geq 1$.

The choice balances the cost of policy evaluation against the number of policy improvements.

# Policy Iteration with Continuous States

For the consumption–saving model, the state $s$ and saving choice $a$ are continuous:

$$V^{\sigma}(s) = u(s - \sigma(s)) + \beta \mathbb{E}[V^{\sigma}(\sigma(s) + y)],$$

$$\widetilde{\sigma}(s) \in \underset{0 \le a \le \min\{s, M\}}{\arg\max} \{u(s - a) + \beta \mathbb{E}[V^{\sigma}(a + y)]\}.$$

Here $y$ is income, and $M$ is the saving limit.

- **Represent the value function:** store values on a grid and interpolate between grid points.
- **Compute expectations:** approximate the income integral using quadrature or simulation.
- **Choose actions:** search an action grid or use continuous optimization.

Solving the resulting finite MDP accurately does not eliminate its grid and integration errors.

# References and Further Reading


1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Chapter 4: Dynamic Programming
2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
   - Discrete State Dynamic Programming — Policy Function Iteration
   - Discrete State Dynamic Programming — Modified Policy Function Iteration